Multimodal table scanning application

Multimodal table scanning application

1. Concept introduction
    1.1 What is "multimodal table scanning"?
    1.2 Brief description of the implementation principle
2. Code analysis
    Key code
        1. The tool layer entry (`largemodel/utils/tools_manager.py`)
        2. The model interface layer (`largemodel/utils/large_model_interface.py`)
    Code analysis
3. Hands-on operation
    3.1 Configure the large model
        3.1.1 Configure the LLM platform (`yahboom.yaml`)
        3.1.2 Configure the model interface (`large_model_interface.yaml`)
    3.2 Start it and test the function (text mode)
    3.3 Voice interaction
4. Frequently asked questions and solutions
        Problem 1: the content of the table is not fully recognised or there are typos.
        Problem 2: it reports "the table file cannot be found"

---

1. Concept introduction

1.1 What is "multimodal table scanning"?

Multimodal table scanning is a technology that uses image processing and artificial intelligence to recognise and extract the information of a table from a picture or a PDF document. It does not only focus on the visual recognition of the table structure; it also combines the text content, the layout information and other multimodal data to improve the understanding of the table. The **large language model (LLM)** in turn provides a powerful semantic analysis capability for understanding this extracted information; the two complement each other and together raise the level of intelligence of the document processing.

1.2 Brief description of the implementation principle

1. **Table detection and content recognition**
    - Use computer vision technology to locate the table in the document and convert the text inside the table into an editable format with OCR technology.
    - Use deep learning methods to parse the structure of the table (the division into rows and columns, the merged cells and so on) and to generate a structured data representation.
2. **Multimodal fusion**
    - Combine the visual information (such as the table layout), the text (the OCR result) and the metadata that may exist (such as the file type and the source) into a comprehensive data view.
    - Use a specifically designed multimodal model (for example LayoutLM) to handle these different kinds of data at the same time, so that the content of the table and its contextual relations are understood more accurately.

2. Code analysis

Key code

1. The tool layer entry (`largemodel/utils/tools_manager.py`)

The `scan_table` function in this file defines the execution flow of the tool, in particular how it builds a Prompt that asks for the result in the Markdown format.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan a table from an image and save the content as a Markdown file.

        :param args: Arguments containing the image path.
        :return: Dictionary with the file path and the content.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args: {args}")
        try:
            image_path = args.get("image_path")
            # ... (the path check and the fallback)

            # Construct a prompt asking the large model to recognise the table and return it
in the Markdown format.
            if self.node.language == 'zh':
                prompt = "Please carefully analyze this image, identify the table within it,
and return its content in Markdown format."
            else:
                prompt = "Please carefully analyze this image, identify the table within it,
and return its content in Markdown format."

            result = self.node.model_client.infer_with_image(image_path, prompt)

            # ... (extracting the Markdown text from the result)

            # Save the recognised content to a Markdown file.
            md_file_path = os.path.join(self.node.pkg_path, "resources_file",
"scanned_tables", f"table_{timestamp}.md")
            with open(md_file_path, 'w', encoding='utf-8') as f:
                f.write(table_content)

            return {
                "file_path": md_file_path,
                "table_content": table_content
            }
        # ... (the error handling)
```

2. The model interface layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all the image related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (preparing the message)
        try:
            # Depending on the value of self.llm_platform, decide which implementation to
call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... (the logic for calling the Tongyi model)
                pass
            # ... (the logic of the other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code analysis

The table scanning function is a typical application of converting unstructured image data into structured text data. Its core technology is still **guiding the behaviour of the model through Prompt Engineering**.

1. **The tool layer (`tools_manager.py`)**:
    - The `scan_table` function is the business flow controller of this function. It receives an image that contains a table as the input.
    - The most critical operation of this function is **building a Prompt with a clear goal**. This Prompt directly tells the large model to perform two tasks: 1. recognise the table in the image. 2. return the recognised content in the Markdown format. This mandatory requirement on the output format is the key to the conversion from unstructured to structured.
    - Once the Prompt has been built, it calls the `infer_with_image` method of the model interface layer and passes the image together with this formatting instruction.
    - After it receives the returned Markdown text from the model interface layer, it performs a file operation: it writes this text content into a new `.md` file.
    - Finally, it returns the structured data that contains the path of the new file and the content of the table.
2. **The model interface layer (`large_model_interface.py`)**:
    - The `infer_with_image` function continues to act as the unified "dispatch centre". It receives the image and the Prompt from `scan_table` and dispatches the task to the correct backend model implementation according to the current system configuration (`self.llm_platform`).
    - Whatever the backend model is, the task of this layer is to handle the communication details of the specific platform, to make sure that the image and the text data are sent correctly, and then

to hand the plain text returned by the model (here the text in the Markdown format) back to the tool layer.

In summary, the general flow of the table scanning is: `ToolsManager` receives the image and builds an instruction to "turn the table in this picture into Markdown" -> `ToolsManager` calls the model interface -> `model_interface` packs the image and that instruction together and sends it to the corresponding model platform according to the configuration -> the model returns the text in the Markdown format -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` saves the text as a `.md` file and returns the result. This flow shows how the format following ability of the large model can be used as a powerful OCR (optical character recognition) and data structuring tool.

3. Hands-on operation

3.1 Configure the large model

This section selects the backend through the platform option in `yahboom.yaml` and fills in the corresponding model parameters in `large_model_interface.yaml`. The online example uses Tongyi Qianwen; the offline example uses the local Ollama.

3.1.1 Configure the LLM platform (`yahboom.yaml`)

Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set `llm_platform` in the existing `model_service.ros__parameters` parameter block. You only have to choose one of the two backends:

```
model_service:
  ros__parameters:
    language: 'en'
    llm_platform: 'ollama'  # The offline large model; change it to 'tongyi' when you use
the online model
    regional_setting: "international"
```

- Offline Ollama: set `llm_platform: 'ollama'`.
- Online Tongyi Qianwen: set `llm_platform: 'tongyi'`.

If these parameters already exist in the file, only modify the corresponding values and keep the other configuration; do not add duplicate YAML keys with the same name.

3.1.2 Configure the model interface (`large_model_interface.yaml`)

Open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Depending on the backend selected in the previous step, only fill in the matching group of parameters.

Online Tongyi Qianwen: first apply for the API Key as described in the online large model basic tutorial and confirm that the account has available quota; then fill in the Key and a model that supports image input:

```
qianwen_api_key: "Replace it with your own API-Key"
qianwen_model: "qwen-vl-max-latest"
```

Replace the placeholder with your own API Key. The table scanning needs a model that is able to read the text and the table structure in the image and to organise it into Markdown following the prompt. Never publish a real API Key.

Offline Ollama: make sure the Ollama service is running and that a model supporting image input has been downloaded; then fill in the service address and the name of the local model:

```
ollama_host: "http://localhost:11434"
ollama_model: "llava"  # Replace it with the model name from ollama list
```

The model name has to match the output of `ollama list`, and the model has to support image input.

After saving both configuration files, restart `largemodel` so that the settings take effect. The online mode requires the Jetson to be able to reach the Tongyi service; the offline mode requires the local Ollama service to be available.

> The speed of the table recognition of the offline large model depends on the computing power of the Jetson mainboard and is usually slower than the online large model; the actual speed also depends on the size of the model and on the clarity of the picture.

3.2 Start it and test the function (text mode)

1. **Prepare the table image file**:

   Put a table image file to be tested at the following path:
   `/home/jetson/yahboom_ws/src/largemodel/resources_file/scan_table`

   Then name the image `test_table.jpg`

2. **Start the `largemodel` main program (text mode)**:

   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

3. **Send the text instruction**:
   Open another terminal and run the following command,

   ```
   ros2 run text_chat text_chat
   ```

   Then start typing the text: "analyse the table".

4. **Watch the result**:
   In the first terminal that runs the main program you will see the log output showing that the system received the instruction, called the `scan_table` tool, reported that the scan_table execution completed and saved the scanned information to a document.

You can find this document at the path ~/yahboom_ws/src/largemodel/resources_file/scan_table

3.3 Voice interaction

The voice mode can use the online or the offline LLM. In `yahboom.yaml`, `llm_platform` is set to `tongyi` for the online and to `ollama` for the offline; the ASR/TTS are chosen independently:

```
model_service:
  ros__parameters:
    use_oline_asr: False  # True: online ASR; False: offline ASR
    useolinetts: False    # True: online TTS; False: offline TTS
```

Complete the corresponding configuration as described in the ASR/TTS tutorials. Confirm that the table image is in the directory specified above and named `test_table.jpg`, and then start it:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, say "analyse the table". Check the recognised text of the ASR, the log of the `scan_table` tool call and the saved Markdown file, and confirm that the speaker announces it. The tool call depends on the current version of the program and on the capability of the model; if the tool was not executed, verify it with the text mode first.

4. Frequently asked questions and solutions

Problem 1: the content of the table is not fully recognised or there are typos.

Solution:

1. **The image quality**: make sure the input table image is sharp, not distorted and evenly lit. A low quality image is the most common cause of the recognition errors.
2. **The model choice**: a model with more parameters recognises better; try switching to a stronger recognition model.

Problem 2: it reports "the table file cannot be found"

Solution:

1. **Check the path**: make sure you have placed the table image file at the specified path, named it `test_table.jpg` and that you have the permission to read the file.
2. **The image format**: make sure the image file is not corrupted and that it is in the .jpg format.